Buscaminas en MARIE

Nicolás Naranjo[1], Juan David Burgos[1], Felipe Bohórquez[1]
[1]Universidad San Francisco de Quito, POLI

MARIE.js, Assembly, Buscaminas, Programación

Introducción

MARIE.js es un simulador de arquitectura educativa que permite ejecutar programas en lenguaje ensamblador sobre una arquitectura von Neumann básica.

Este proyecto implementa el juego **Buscaminas** en un tablero de 16×16 celdas, donde la interacción con el usuario se realiza mediante ingreso secuencial de coordenadas (fila, columna, acción).

El tablero inicial es generado por un script en **Python** que produce el mapa de minas y valores en el formato requerido por MARIE.

Metodología

La metodología consiste en dos partes:

1. Generación del tablero con Python:
- Define coordenadas de minas
- Calcula valores de celdas vecinas
- Genera el bloque de memoria

2. Lógica del juego en MARIE:
- Valida coordenadas ingresadas
- Actualiza el tablero visible
- Verifica victoria o derrota

El flujo del juego opera en un ciclo continuo: usuario ingresa tres valores (fila, columna, acción), programa valida, ejecuta acción, actualiza display y verifica si terminó.

Entrada del Usuario

El usuario interactúa ingresando tres valores:

- **Fila**: Índice de fila (0-15)
- **Columna**: Índice de columna (0-15)
- **Acción**:
  - 1 = revelar celda
  - 2 = marcar/desmarcar bandera

El programa valida que las coordenadas estén en rango válido antes de ejecutar cualquier acción.

Display 16×16

El display usa direccionamiento lineal (0-255) donde cada pixel representa una celda del tablero.

Asignación de colores:

Oculta	2108 (gris oscuro)
Bandera	7FE0 (amarillo)
Mina	Color especial
Revelada	Colores según valor de minas vecinas

Memoria RAM: El tablero se almacena Starting en dirección D, con valores de −1 (mina) a 8 (vecinas).

Algoritmos Principales

VerifyInput: Valida que fila y columna estén en rango 0-15. Retorna "invalid" si están fuera de rango, o "reveal"/"flag" según la acción.

UpdateBoardIdx: Convierte coordenadas 2D (fila, columna) a índice lineal (0-255) multiplicando la fila por 16 y sumando la columna.

ActionReveal: Obtiene el valor real del tablero. Si es −1 (mina) ⟶ pierde. Si es 0 ⟶ ejecuta FloodFill. Actualiza el display.

ActionFlag: Marca/desmarca una celda. Si está oculta ⟶ cambia a amarillo (7FE0). Si ya tiene bandera ⟶ la remueve (2108).

FloodFill: Cuando se revela una celda vacía (valor 0), busca y revela automáticamente todas las celdas vacías adyacentes.

CheckVictory: Compara celdas reveladas (sr) con el total de celdas sin mina (SafeTiles). Si son iguales ⟶ victoria.

VerifyInput

Algoritmo 1: VerifyInput

```
▷ r: row, c: column, a: action
function VerifyInput(r, c, a)
    if r < 0 then
        return invalid
    end
    if c < 0 then
        return invalid
    end
    if r ≥ 16 then
        return invalid
    end
    if c ≥ 16 then
        return invalid
    end
    if a == 1 then
        return reveal
    else if a == 2 then
        return flag
    end
end
```

UpdateBoardIdx

Algoritmo 2: UpdateBoardIdx

```
▷ r: row index (0-15), c: column index (0-15)
▷ i: linear index result, k: loop counter
function UpdateBoardIdx(r, c)
    i ← 0
    k ← 0
    while r > k do
        i ← i + 16
        k ← k + 1
    end
    i ← i + c
    return i
end
```

ActionReveal

Algoritmo 3: ActionReveal

```
▷ r, c: row/col input, x: linear index
▷ value: tile value (-1=mine, 0-8)
function ActionReveal(r, c, sr)
    x ← UpdateBoardIdx(r, c)
    PixelPtr ← UpdatePixelPtr(x)
    if PixelPtr == 2108 then
        BoardPtr ← D + x
        ColorPtr ← COLOR_ZERO_PTR + *BoardPtr
        *PixelPtr ← *ColorPtr
        value ← *ColorPtr − COLOR_MINE
        if value == −1 then
            return defeat
        end
        sr ← sr + 1
        if value == 0 then
            FloodFill(x)
        end
    end
    return check
end
```

ActionFlag

Algoritmo 4: ActionFlag

```
▷ r, c: row/col input, x: linear index
function ActionFlag(r, c)
    x ← UpdateBoardIdx(r, c)
    PixelPtr ← UpdatePixelPtr(x)
    if PixelPtr == 2108 then
        PixelPtr ← 7FE0
    end
    if PixelPtr == 7FE0 then
        PixelPtr ← 2108
    end
end
```

FloodFill

Algoritmo 5: FloodFill

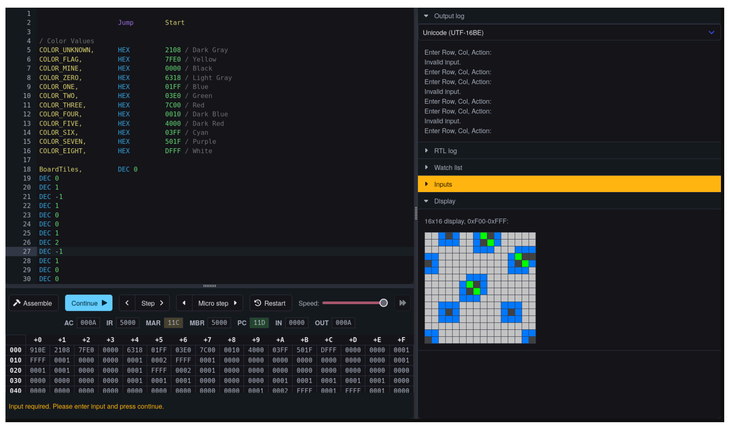
```
▷ x: linear index, rf: repeat flag
function FloodFill(x)
    rf ← 0
    BoardPtr ← D
    PixelPtr ← F00
    while PixelPtr < 1000 do
        if PixelPtr == 2108 then
            if BoardPtr == 0 then
                GetAdjacentTiles(x)
            end
        end
        BoardPtr ← BoardPtr + 1
        PixelPtr ← PixelPtr + 1
    end
    if rf == 1 then
        FloodFill(x)
    end
end
```

CheckVictory

Algoritmo 6: CheckVictory

```
▷ SafeTiles: total non-mine tiles
▷ sr: revealed count
function CheckVictory(SafeTiles, sr)
    if SafeTiles > sr then
        return continue
    end
    if SafeTiles ≤ sr then
        return victory
    end
end
```

Resultados



Conclusiones

La implementación del juego Buscaminas en MARIE demuestra que es posible desarrollar aplicaciones interactivas relativamente complejas en una arquitectura educativa limitada.

El uso de un script externo en Python para generar el tablero permite separar la generación de datos de la lógica del juego, siguiendo un patrón común en sistemas reales.

La visualización mediante colores en el display de 16×16 resulta efectiva para representar los estados del juego, aunque limita la capacidad de mostrar números directamente al usuario.